**Oban vs. Sidekiq**



Ruby's Sidekiq library has been around for a while, and Ruby developers as a whole (myself included) are pretty comfortable with it. It's incredibly stable in production and can process a large throughput of jobs each second. Sidekiq uses Redis to store and process jobs, so I was a bit skeptical at first when I learned that Oban operates on top of a Postgres database.

While there are differences in how Redis is operated versus Postgres, it's not difficult to use Oban at scale. Overall, the Elixir community's experience with Oban has proven that it's a stable job system that can process a large throughput of jobs. The added benefits of transactions and not needing to run a Redis server make it a compelling job system.

Next, let's implement an Oban worker in our SMS app.

## Implement an Oban Worker

In the previous section, you got a sneak peek of what an Oban worker module looks like—it's really simple to create one. But before we get to that point, we need to set up Oban. Once Oban is set up for our app, we'll add our worker and test it out.

### Add Oban to Our SMS App

Oban has great guides to walk you through implementing it. It's a fairly quick process, but it might change slightly over time as the library grows and changes. So if you do run into any issues, then make sure to read the Oban Installation Guide.[5]

The first step is to include the Oban dependency in mix.exs:

phone_app/mix.exs
```
{:oban, "~> 2.16"},
```

And then run mix deps.get to bring it into your project.

Oban uses a database to store jobs, so next you need to set up the migrations for it. Start by creating a new migration:

```
$ mix ecto.gen.migration add_oban_jobs_table
```

Then enter the following into the migration file. Your file name will vary because it's based on the time that you run the migration command:

5. https://hexdocs.pm/oban/installation.html